Water Jug Problem Solver

State-Space Search using Breadth-First Search (BFS) and Depth-First Search (DFS)

Course Project Report
Student: _____
Register Number: _____
Project: Water Jug Problem Solver

1. Problem Formulation

The Water Jug Problem is a classical artificial intelligence state-space search problem. Two jugs with fixed capacities A and B are initially empty. The objective is to obtain a target quantity of water in either jug by applying a sequence of legal operations.

State Representation: A state is represented as (x, y), where x is the current amount of water in Jug A and y is the current amount of water in Jug B. The initial state is (0, 0).

Actions: The legal actions are Fill A, Fill B, Empty A, Empty B, Pour A to B, and Pour B to A. Every action transforms the current state into a new valid state.

Goal Test: A state is considered a goal when the target amount exists in either jug. Therefore, the goal condition is x = target or y = target.

Path Cost: Each operation has a uniform cost of one. Therefore, the total path cost equals the number of operations used to reach the goal.

2. Modelling Actions as State Transitions

Action	State Transition
Fill A	(x, y) -> (A, y)
Fill B	(x, y) -> (x, B)
Empty A	(x, y) -> (0, y)
Empty B	(x, y) -> (x, 0)
Pour A -> B	Transfer water until A is empty or B is full
Pour B -> A	Transfer water until B is empty or A is full

3. Approach and Algorithms

Breadth-First Search (BFS): BFS explores the state space level by level using a FIFO queue. Since every operation has the same cost, BFS guarantees that the first solution found contains the minimum possible number of operations. A visited set prevents repeated exploration of states, while parent pointers are used to reconstruct the ordered solution path.

Depth-First Search (DFS): DFS explores one branch of the state space as deeply as possible before backtracking. It uses a LIFO stack. DFS may reach a solution while exploring fewer states, but it does not guarantee the shortest sequence of operations.

Both algorithms demonstrate the concept of modelling a problem as a graph where each node is a state and each edge is a legal action. The final output reconstructs the path from the start state to the goal state.

4. Complexity Analysis

For jug capacities A and B, the maximum number of possible states is S = (A + 1)(B + 1). Each state has at most six possible actions. Therefore, in the bounded state-space graph, both BFS and DFS have time complexity O(S) and space complexity O(S). The practical performance depends on the order in which states are explored.

Algorithm	Time Complexity	Space Complexity	Guarantee
BFS	O(S)	O(S)	Shortest path
DFS	O(S)	O(S)	Solution not always shortest

5. Experimental Results and Comparison

The solver was evaluated on multiple Water Jug Problem configurations. The benchmark compares BFS and DFS using three main metrics: number of solution steps, nodes expanded, and execution runtime.

Problem	Method	Steps	Nodes Expanded	Runtime (ms)
classic-4-3-2	BFS	4	10	0.1321
classic-4-3-2	DFS	6	8	0.0277
5-3-4	BFS	6	13	0.0196
5-3-4	DFS	6	8	0.0136
7-4-6	BFS	6	13	0.0166
7-4-6	DFS	6	8	0.0128
8-5-6	BFS	6	13	0.0165
8-5-6	DFS	6	8	0.0127
9-4-7	BFS	10	22	0.0347
9-4-7	DFS	14	16	0.0264

Figure 1: Comparison of Nodes Expanded

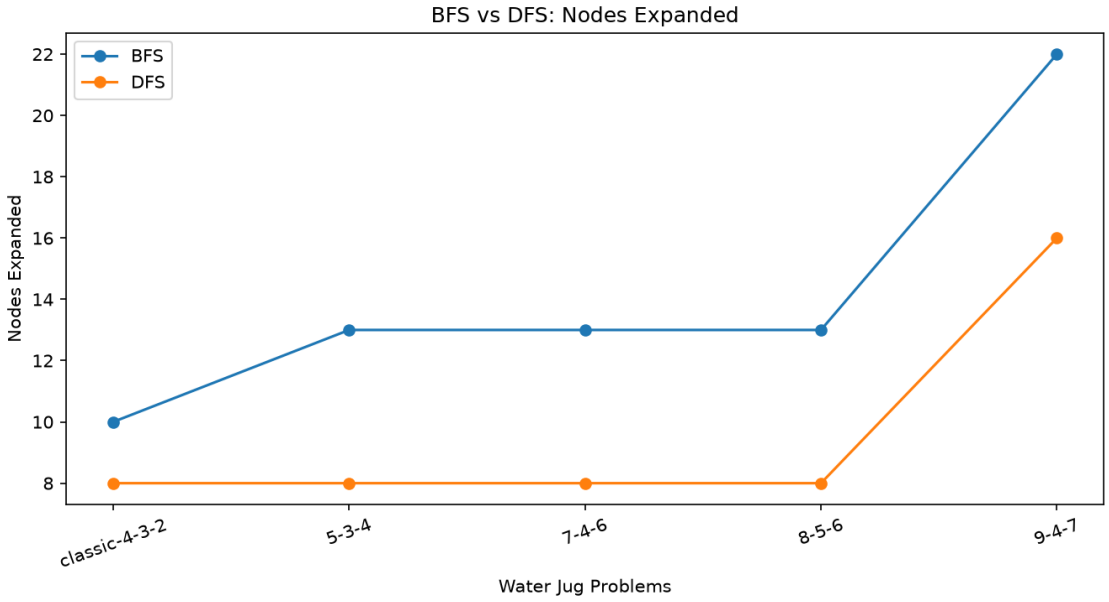
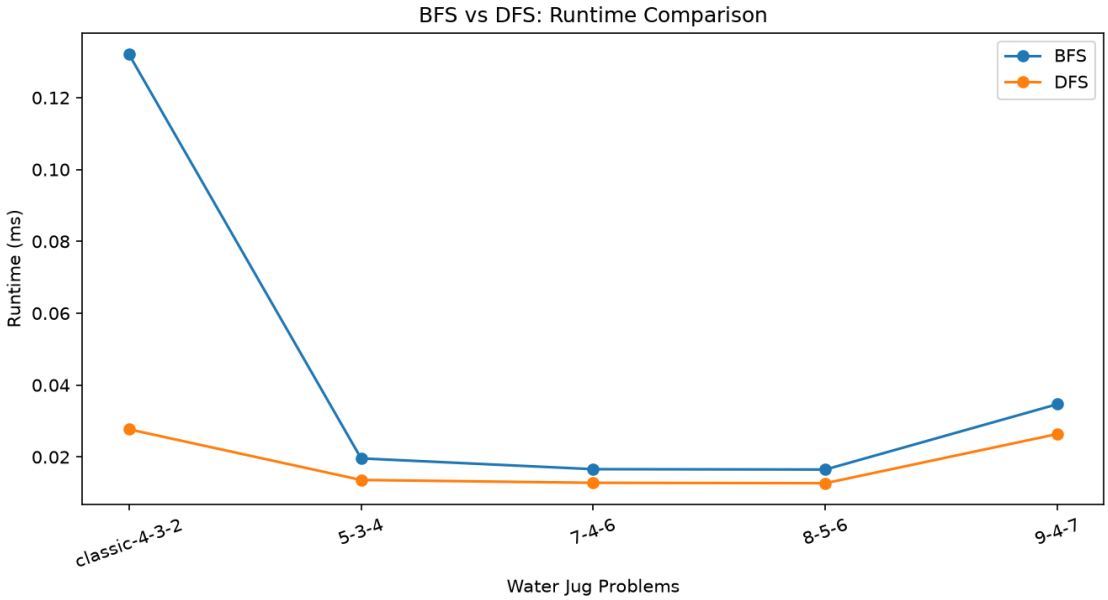


Figure 2: Runtime Comparison



Interpretation of Results

The results show that BFS consistently provides an optimal solution in terms of the number of operations because it explores states level by level. DFS can sometimes expand fewer nodes and execute faster, but its solution may contain more steps. Therefore, BFS is preferred when the shortest sequence of operations is important, while DFS can be useful when memory or quick exploration is the primary concern.

6. Reflection and Future Improvements

The most challenging part of the project was correctly representing the Water Jug Problem as a finite state-space graph and ensuring that repeated states did not create cycles. The use of a visited set solved this issue, while parent pointers made it possible to display the ordered sequence of operations.

Future improvements could include an interactive graphical interface, animation of water transfers, support for more than two jugs, and additional search algorithms such as Uniform Cost Search or A*. Another improvement would be allowing users to enter custom jug capacities and targets directly through a user interface.

Honesty Note: AI tools were used for guidance and documentation assistance. The algorithms, code structure, execution, testing, and project understanding were reviewed and verified by the student.